

Samuel Oliveira

Ph0enix



Github



LinkedIn



Website



EyeWeb Relatório Técnico

Informações do documento:

Título	EyeWeb: Plataforma de Verificação de Violações de Dados
Cliente	ISTEC Porto: Instituto Superior de Tecnologias Avançadas do Porto
Autor	Samuel Oliveira
Data de realização	Abril 2026
Versão	1.0
Classificação	Público

Controlo de Versões:

Versão	Data	Autor	Descrição
1.0	Abril - 2026	Samuel Oliveira	Versão final do relatório

Distribuição:

Nome	Cargo	Organização
José Oliveira	Desenvolvedor	ISTEC Porto
Ana Rita Monteiro	Desenvolvedor	ISTEC Porto
Vanina Kollen	Desenvolvedor	ISTEC Porto
Tiago Carvalho	Desenvolvedor	ISTEC Porto
Francisco Ribeiro	Analista de Sistemas	ISTEC Porto



Índice

Introdução.....	5
Âmbito e Autorização.....	6
Metodologia de Análise.....	7
Arquitetura Geral.....	7
Modelo de Privacidade, K-Anonymity.....	8
Endpoints da API	9
Armazenamento de Dados.....	10
Base de Dados Supabase (PostgreSQL)	11
Sistema de Autenticação.....	12
Segurança da Plataforma.....	13
Infraestrutura e Deployment.....	15
Plano de Melhoria.....	16
Conclusão.....	17
Referências	18
Glossário	19



Índice de imagens

Figura 1: Visão geral da arquitetura do EyeWeb.....	7
Figura 2: Fluxo do modelo de privacidade K-Anonymity.	8
Figura 3: Comunicação entre componentes na análise de URL.....	10
Figura 4: Diagrama entidade-relacionamento do Supabase com RLS.	11
Figura 5: Autenticação por email e MFA (TOTP) para administradores.	12
Figura 6: Infraestrutura de produção e integração de serviços.....	15



Introdução

O presente relatório documenta de forma detalhada o projeto EyeWeb, uma plataforma de cibersegurança desenvolvida como Projeto Final do 2.º ano do CTeSP em Cibersegurança no Instituto Superior de Tecnologias Avançadas do Porto (ISTEC), ano letivo 2025/2026.

O EyeWeb permite a qualquer utilizador verificar se os seus dados pessoais: endereços de email, números de telefone e passwords que foram expostos em violações de dados conhecidas (data breaches). Adicionalmente, oferece análise inteligente de URLs potencialmente maliciosos, identificando ameaças como phishing, malware e engenharia social.

A motivação principal do projeto reside na necessidade crescente de ferramentas acessíveis que permitam ao cidadão comum proteger a sua presença digital. A plataforma foi desenhada com um compromisso total com a privacidade do utilizador, recorrendo ao modelo matemático de K-Anonymity para garantir que nenhum dado pessoal é exposto ao servidor durante as verificações.

O EyeWeb encontra-se disponível publicamente em <https://eyeweb.vercel.app> e o seu código-fonte é aberto, hospedado em <https://github.com/Sam-Ciber-Dev/eyeweb>.



Âmbito e Autorização

Este projeto foi desenvolvido no âmbito da unidade curricular de Projeto Final do CTeSP de Cibersegurança do ISTECS Porto, sob orientação académica. O projeto foi igualmente apresentado na Joteca, uma organização académica do ISTECS, perante representantes de empresas e outras instituições de ensino superior.

O âmbito do projeto abrange:

- Verificação de emails e números de telefone contra bases de dados de violações de dados públicas, utilizando hashing SHA-256 e o modelo de K-Anonymity;
- Verificação de passwords comprometidas através do mesmo modelo de privacidade;
- Análise de segurança de URLs utilizando a API Google Safe Browsing e inteligência artificial (Groq LLaMA 3.3 70B);
- Chatbot assistente integrado com IA para responder a questões sobre cibersegurança;
- Painel de administração com autenticação multifator (TOTP), monitorização de tráfego em tempo real, deteção de ameaças e gestão de utilizadores bloqueados;
- Sistema de autenticação seguro com verificação por código de email e integração Google OAuth.

Todos os dados de violações utilizados são provenientes de bases de dados públicas e de acesso livre. Nenhum dado pessoal real de utilizadores é armazenado pelo servidor apenas hashes criptográficos são transmitidos.



Metodologia de Análise

A arquitetura do EyeWeb segue o modelo cliente-servidor, com separação clara entre frontend e backend, comunicando exclusivamente via API REST sobre HTTPS.

Arquitetura Geral

O projeto é composto por três camadas principais:

1. Frontend

Aplicação web construída com Next.js 16 (App Router), TypeScript e React, hospedada na Vercel (região CDG1 — Paris). Responsável pela interface do utilizador, cálculo local de hashes SHA-256, e comparação local dos resultados de K-Anonymity.

2. Backend

REST construída com FastAPI (Python 3.11+), hospedada no Render.com (região Frankfurt). Responsável pela lógica de negócio, consulta de datasets, integração com APIs externas (Google Safe Browsing, Groq AI, URLScan.io) e gestão de tráfego.

3. Base de Dados

Supabase (PostgreSQL gerido) para autenticação, perfis de utilizador, cache de URLs verificados, logs de tráfego e chat de administração. Hugging Face Datasets para armazenamento dos datasets de breaches e passwords em formato Apache Parquet, particionados por prefixo de hash.

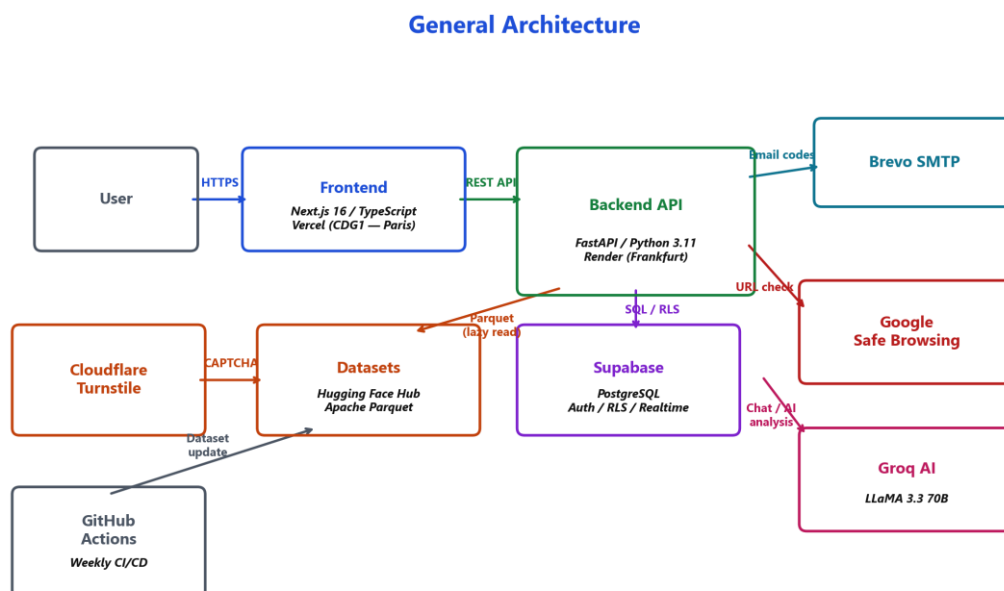


Figura 1: Visão geral da arquitetura do EyeWeb.



Modelo de Privacidade, K-Anonymity

O K-Anonymity é o pilar central da arquitetura de privacidade do EyeWeb. O fluxo funciona da seguinte forma:

1. O utilizador introduz o seu email, telefone ou password no browser;
2. O browser calcula o hash SHA-256 do dado localmente, sem enviá-lo ao servidor;
3. São extraídos apenas os primeiros 5 caracteres do hash (o prefixo);
4. O prefixo é enviado ao backend via HTTPS;
5. O backend retorna TODOS os hashes armazenados que começam com esse prefixo;
6. O browser compara localmente se o hash completo do utilizador está na lista de candidatos;
7. O servidor nunca conhece qual registo específico foi verificado.

Esta abordagem garante privacidade matemática: mesmo que o tráfego de rede fosse interceptado, o atacante saberia apenas o prefixo de 5 caracteres, que corresponde a centenas ou milhares de registos diferentes.

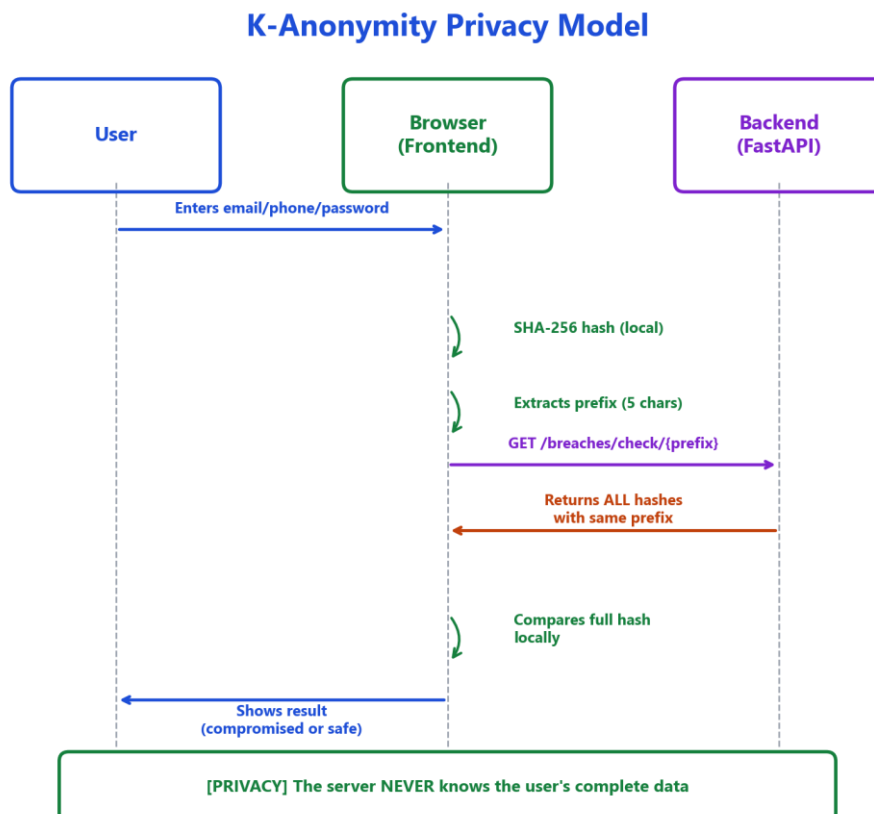


Figura 2: Fluxo do modelo de privacidade K-Anonymity.



Endpoints da API

A API está organizada nos seguintes grupos de endpoints:

GET /api/v1/breaches/check/{prefix}

Verifica emails e telefones contra o dataset de violações. Aceita um prefixo hexadecimal de 2 a 6 caracteres e retorna todos os candidatos correspondentes, incluindo nome da violação, data, e tipos de dados expostos.

GET /api/v1/passwords/check/{prefix}

Verifica passwords comprometidas. Funciona de forma idêntica ao endpoint de breaches, mas consulta o dataset de passwords separado. Retorna hashes candidatos com frequência de ocorrência.

POST /api/v1/urls/check

Analisa a segurança de um URL. Recebe o URL completo, normaliza-o (bloqueia protocolos perigosos como javascript:, data:, file:; bloqueia IPs privados; adiciona HTTPS se necessário), e realiza verificações com Google Safe Browsing e análise de IA. Implementa cache stale-while-revalidate: resultados frescos (< 1 hora) são retornados imediatamente; resultados antigos (1h-30 dias) são retornados do cache enquanto uma re-verificação é iniciada em background; URLs sem cache são verificados em tempo real.

POST /auth/send-code

Gera e envia código de verificação por email via Brevo SMTP. A sessão expira em 5 minutos.

POST /auth/verify-code

Valida o código selecionado pelo utilizador. Em caso de sucesso, o frontend obtém um token JWT do Supabase para acesso autenticado.

POST /admin/verify-mfa

Verifica código TOTP de 6 dígitos para acesso ao painel de administração. Utiliza HMAC-SHA256 com intervalo de 30 segundos e uma janela de tolerância de 2m. Duas tentativas falhadas resultam em bloqueio de 72 horas por IP e device fingerprint.

POST /user/chat/chat

Chatbot público integrado com Groq LLaMA 3.3 70B. Inclui proteções contra prompt injection (regex para XSS, SQL injection, jailbreak) e validação de output (verifica se a resposta contém keywords relevantes ao EyeWeb).

GET /admin/traffic/*

Conjunto de endpoints protegidos para monitorização de tráfego: estatísticas do dashboard, conexões ativas em tempo real, logs paginados, atividade suspeita, e gestão de IPs/devices bloqueados.



API Endpoints

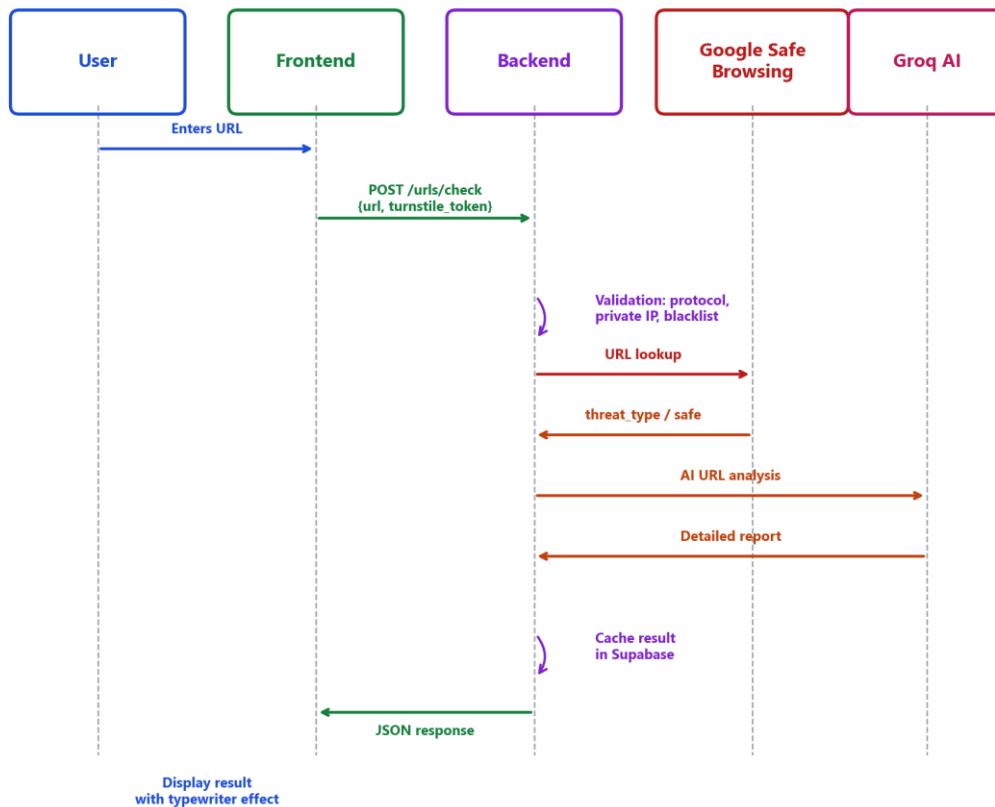


Figura 3: Comunicação entre componentes na análise de URL.

Armazenamento de Dados

Os datasets de violações e passwords são armazenados no Hugging Face Datasets em formato Apache Parquet, particionados em 256 ficheiros (00.parquet até ff.parquet) pelo prefixo de 2 caracteres do hash SHA-256. Esta estratégia permite:

Leitura lazy

Apenas a partição necessária é descarregada, otimizando o uso de memória no free tier do Render (512 MB RAM);

Cache LRU com TTL

Até 100 partições são mantidas em memória durante 1 hora, evitando downloads repetidos;

Escalabilidade

Novos registos são adicionados via workflow GitHub Actions que executa semanalmente, descarregando novos dados e atualizando os ficheiros Parquet no repositório Hugging Face.



Base de Dados Supabase (PostgreSQL)

O Supabase serve como base de dados relacional para dados operacionais:

- **Profiles:** Dados do utilizador (id, email, role, mfa_secret);
- **verification_codes:** Códigos de autenticação temporários (session_id, correct_code, expires_at);
- **url_scans:** Cache de URLs verificados (url_hash, original_url, status, ai_opinion, threat_details, last_check);
- **traffic_logs:** Logs de visitas com geolocalização, deteção de VPN, e device fingerprint (retenção de 30 dias, limpeza automática à meia-noite UTC);
- **blocked_ips / blocked_devices:** Registos de bloqueio manual ou automático;
- **admin_chat_messages:** Histórico do chat de administração com suporte a Supabase Realtime.

Todas as tabelas sensíveis possuem Row-Level Security (RLS) ativada, garantindo que utilizadores normais apenas acedem aos seus próprios dados.

Supabase Database (PostgreSQL)

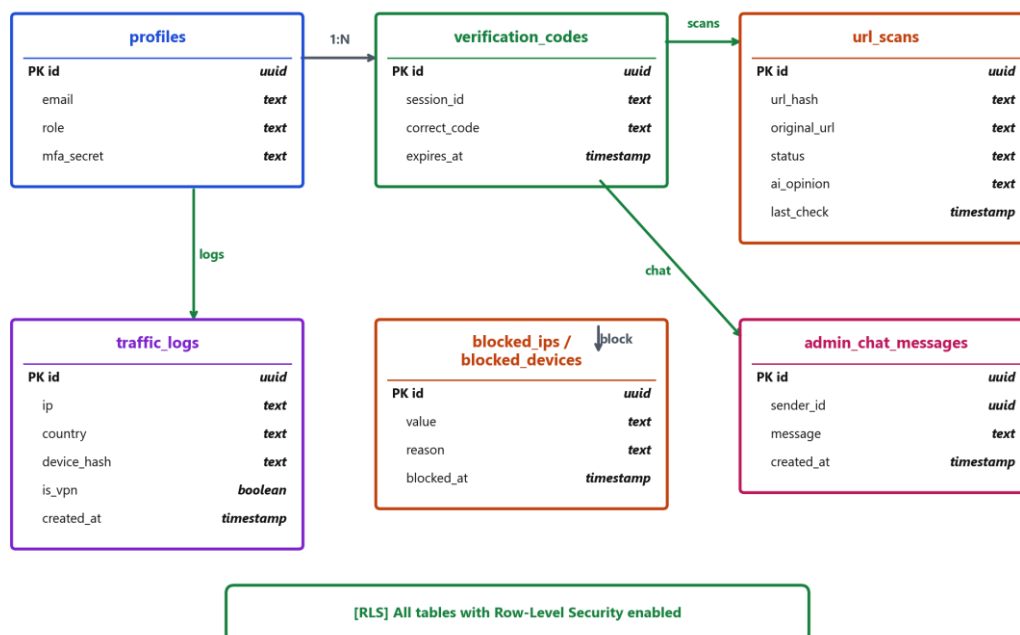


Figura 4: Diagrama entidade-relacionamento do Supabase com RLS.



Sistema de Autenticação

A autenticação do EyeWeb combina múltiplos mecanismos:

Login por código de email

O utilizador recebe 3 códigos por email (via Brevo SMTP), sendo apenas um válido. Esta abordagem anti-phishing dificulta ataques de reutilização de códigos.

Google OAuth

Login via conta Google, com registo automático de perfil.

MFA para administradores

Códigos TOTP de 6 dígitos gerados offline por uma aplicação desktop Python (eyeweb_auth.py). Os códigos nunca transitam pela rede; a verificação é feita por comparação de HMAC-SHA256 no servidor.

Tokens JWT

Após autenticação, o Supabase emite tokens JWT que são validados em cada pedido protegido. O backend verifica o token com o Supabase e compara o hash SHA-256 do email com a variável de ambiente ADMIN_EMAIL_HASH para confirmar acesso administrativo.

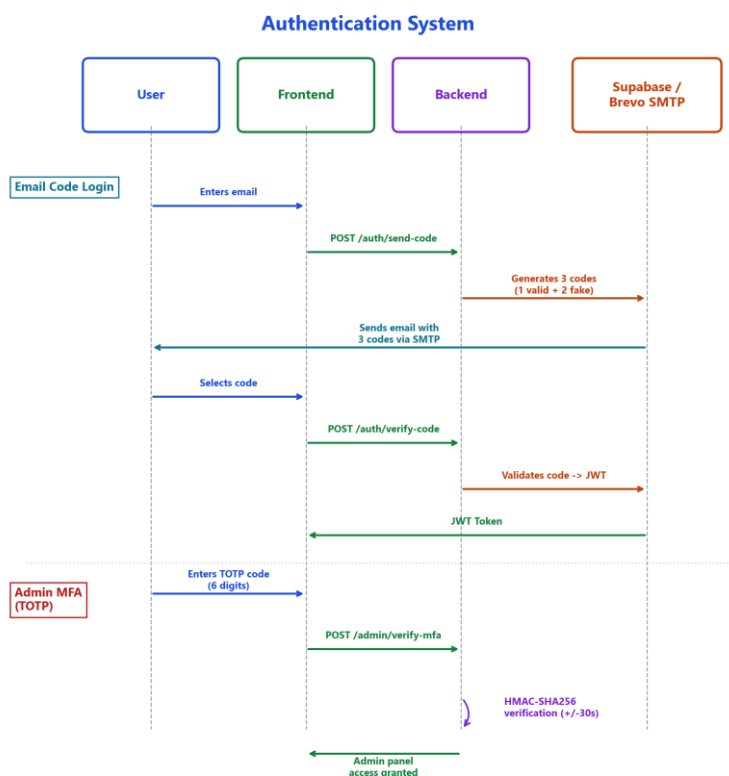


Figura 5: Autenticação por email e MFA (TOTP) para administradores.



Segurança da Plataforma

As seguintes medidas de segurança estão implementadas:

- **CORS restritivo:** Em produção, apenas `https://eyeweb.vercel.app` é permitido; em desenvolvimento, `localhost` é adicionado;
- **Swagger/Docs desativados em produção:** Os endpoints `/docs`, `/redoc` e `/openapi.json` retornam 404 quando `ENVIRONMENT=production`;
- **Rate limiting:** 60 pedidos por minuto por IP nos endpoints públicos;
- **Cloudflare Turnstile:** CAPTCHA no formulário de registo/login para prevenir bots;
- **Validação de URLs:** Bloqueio de protocolos maliciosos (`javascript:`, `data:`, `vbscript:`, `file:`), IPs privados (`127.x`, `192.168.x`, `10.x`, `172.16-31.x`) e protocolos malformados (`ht+ps`, `https`);
- **Proteção contra prompt injection no chatbot:** Regex para detetar XSS, SQL injection, jailbreak, DAN mode e outras tentativas de manipulação. Validação dupla: input do utilizador E output da IA;
- **Headers de segurança (Vercel):** `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `X-XSS-Protection: 1; mode=block`;
- **Device fingerprinting:** Canvas, WebGL, Audio API, resolução de ecrã, CPU, RAM, timezone, plataforma. Hardware-only hash para detetar o mesmo dispositivo em browsers diferentes. `Score ≥ 70/100` = mesmo dispositivo;
- **Deteção de ameaças em tempo real:** Identificação automática de scanners (Nmap, SQLMap, Nikto, Burp Suite), tentativas de SQL injection, path traversal, e acessos a caminhos suspeitos (`/wp-admin`, `/.env`, `/phpmyadmin`);
- **Bloqueio automático e manual:** IPs e devices podem ser bloqueados automaticamente por comportamento suspeito ou manualmente pelo administrador;
- **Containerização Docker:** O backend executa como utilizador não-root (appuser) dentro de um contentor Python 3.11-slim, sem cache de pip.
- **Frontend Interface do Utilizador:** O frontend é uma aplicação Next.js 16 com App Router, escrita em TypeScript e React. As principais funcionalidades incluem:
- **Quatro verificadores integrados:** Email, Telefone, Password e URL, organizados em abas;
- **Cálculo local de hashes:** Toda a criptografia SHA-256 é executada no browser do utilizador, garantindo que os dados brutos nunca saem do dispositivo;
- **Medidor de força de password:** Avaliação em tempo real baseada em comprimento, diversidade de caracteres e anti-padrões (repetições, apenas letras);
- **Efeito typewriter na análise de IA:** Os resultados da análise de URL são apresentados com animação de escrita, criando uma experiência interativa;
- **Chatbot Eye:** Widget flutuante que permite aos utilizadores fazer perguntas sobre cibersegurança em português ou inglês;
- **Internacionalização completa (PT/EN):** Mais de 300 chaves de tradução, com deteção automática do idioma do browser;



- **Painel de administração:** Dashboard com gráficos (Recharts) de tráfego, mapa de países, timeline de eventos (estilo Wireshark), detecção de VPN, e chat de administração em tempo real (Supabase Realtime);
- **Tracking de visitas:** Componente invisível que regista visitas e envia heartbeats a cada 5 minutos com device fingerprint;
- **Middleware de bloqueio:** Antes de cada página carregar, verifica se o IP ou device fingerprint do visitante está bloqueado;
- **Animações com Framer Motion:** Transições suaves entre componentes e páginas;
- **Páginas legais:** Política de Privacidade e Termos de Serviço, acessíveis pelo footer.



Infraestrutura e Deployment

Frontend: Hospedado na Vercel, região CDG1 (Paris), com auto-deploy a partir do branch main do repositório GitHub. Build Next.js estático + SSR;

Backend: Hospedado no Render.com, região Frankfurt, com auto-deploy via Docker. Plano free tier (512 MB RAM, spin-down após inatividade);

Base de Dados: Supabase free tier, com autenticação, PostgreSQL, Realtime e Storage;

Datasets: Hugging Face Hub, repositórios privados com token de leitura;

CI/CD: GitHub Actions: workflow semanal para atualização do dataset de breaches (actions/checkout@v6, actions/setup-python@v6);

SMTP: Brevo (smtp-relay.brevo.com:587) para envio de códigos de verificação;

DNS: Domínio gerido pela Vercel (eyeweb.vercel.app).

Infrastructure & Deployment

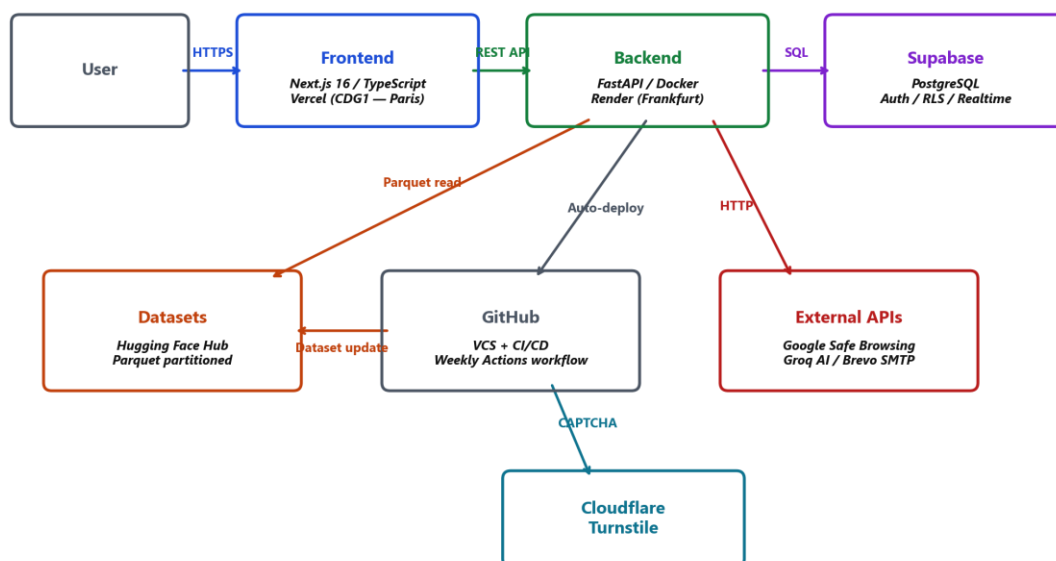


Figura 6: Infraestrutura de produção e integração de serviços.



Plano de Melhoria

Com base na análise técnica realizada, recomenda-se para evolução futura do projeto:

- 1. Migração para planos pagos:** O free tier do Render resulta em spin-down após inatividade, causando cold starts de ~30 segundos. Um plano pago eliminaria esta latência e permitiria mais RAM para cache de partições.
- 2. Ativação da proteção contra passwords comprometidas no Supabase:** Esta funcionalidade está disponível apenas no plano Pro do Supabase. No entanto, o EyeWeb já implementa a sua própria verificação de passwords via K-Anonymity, o que mitiga parcialmente esta limitação.
- 3. Adição de mais fontes de dados de violações:** Integrar bases de dados adicionais (e.g., Have I Been Pwned API, BreachDirectory) para aumentar a cobertura de deteção.
- 4. Implementação de notificações:** Permitir que utilizadores registados recebam alertas automáticos quando os seus dados aparecem em novas violações.
- 5. Testes automatizados:** Implementar suíte de testes unitários e de integração com pytest (backend) e Jest/Testing Library (frontend) para garantir estabilidade em futuras atualizações.
- 6. Auditoria de segurança externa:** Submeter a plataforma a um pentest formal por uma entidade externa para identificar vulnerabilidades não detetadas na auditoria interna.
- 7. Certificado SSL personalizado:** Embora a Vercel e o Render forneçam HTTPS automático, um certificado dedicado reforçaria a confiança dos utilizadores.
- 8. Monitorização avançada:** Integrar ferramentas como Sentry (error tracking) e UptimeRobot (disponibilidade) para monitorização contínua em produção.



Conclusão

O EyeWeb cumpre os objetivos propostos como Projeto de Aplicação Prática do CTeSP de Cibersegurança: uma plataforma funcional, segura e acessível que permite a qualquer utilizador verificar se os seus dados pessoais foram comprometidos em violações de dados conhecidas.

Os pilares que sustentam o projeto são:

- **Privacidade por design:** O modelo de K-Anonymity garante que nenhum dado pessoal do utilizador é exposto ao servidor. Toda a comparação de hashes é realizada localmente no browser, tornando a plataforma matematicamente privada.
- **Arquitetura moderna:** A separação entre frontend (Next.js/TypeScript na Vercel) e backend (FastAPI/Python no Render) permite escalabilidade independente, manutenção simplificada e deployment contínuo.
- **Segurança em camadas:** Desde a validação de inputs e proteção CORS, passando pela deteção de ameaças em tempo real e device fingerprinting, até ao bloqueio automático de atacantes e autenticação multifator para administradores.
- **Inteligência artificial integrada:** A análise de URLs com Groq LLaMA 3.3 70B e o chatbot assistente demonstram a aplicação prática de IA na cibersegurança, com proteções contra prompt injection implementadas.

O projeto demonstra que é possível construir ferramentas de cibersegurança robustas e privadas utilizando exclusivamente serviços gratuitos (Vercel, Render, Supabase, Hugging Face, Groq), tornando-as acessíveis para fins académicos e educativos.

A apresentação do EyeWeb na Joteca do ISTECS Porto validou a relevância do projeto perante representantes de empresas e instituições de ensino superior, confirmando o seu valor como ferramenta de sensibilização para a segurança de dados pessoais.



Referências

- **FastAPI:** <https://fastapi.tiangolo.com/>
- **Next.js:** <https://nextjs.org/>
- **Supabase:** <https://supabase.com/>
- **Hugging Face Datasets:** <https://huggingface.co/docs/datasets/>
- **Groq:** <https://groq.com/>
- **Google Safe Browsing API:** <https://developers.google.com/safe-browsing/>
- **K-Anonymity:** <https://dataprivacylab.org/dataprivacy/projects/kanonymity/>
- **Have I Been Pwned:** <https://haveibeenpwned.com/>
- **Cloudflare Turnstile:** <https://developers.cloudflare.com/turnstile/>
- **Apache Parquet:** <https://parquet.apache.org/>
- **Render:** <https://render.com/>
- **Vercel:** <https://vercel.com/>
- **Brevo (Sendinblue) SMTP:** <https://www.brevo.com/>
- **Docker:** <https://www.docker.com/>
- **OWASP Top 10:** <https://owasp.org/www-project-top-ten/>



Glossário

- **API (Application Programming Interface):** Interface que permite a comunicação entre diferentes sistemas de software.
- **CORS (Cross-Origin Resource Sharing):** Mecanismo de segurança que restringe pedidos HTTP entre domínios diferentes.
- **CSRF (Cross-Site Request Forgery):** Ataque que força um utilizador autenticado a executar ações indesejadas.
- **CTeSP:** Curso Técnico Superior Profissional, formação de nível 5.
- **Data Breach:** Incidente de segurança em que dados pessoais são acedidos, divulgados ou roubados sem autorização.
- **Device Fingerprint:** Identificador único de um dispositivo baseado nas suas características de hardware e software.
- **FastAPI:** Framework web moderno para Python, otimizado para APIs RESTful de alta performance.
- **Hash (SHA-256):** Função criptográfica que transforma dados de qualquer tamanho num valor fixo de 256 bits, irreversível.
- **HMAC:** Hash-based Message Authentication Code, mecanismo de autenticação de mensagens baseado em hash.
- **JWT (JSON Web Token):** Formato de token compacto para transmissão segura de informação entre partes.
- **K-Anonymity:** Modelo de privacidade que garante que cada registo é indistinguível de pelo menos K-1 outros registos.
- **LLaMA:** Large Language Model Meta AI, modelo de linguagem de grande escala desenvolvido pela Meta.
- **MFA (Multi-Factor Authentication):** Autenticação que requer dois ou mais fatores de verificação.
- **Next.js:** Framework React para aplicações web com renderização no servidor e geração estática.
- **Parquet:** Formato de armazenamento colunar otimizado para grandes volumes de dados.
- **PAP:** Projeto de Aplicação Prática, projeto final do CTeSP.
- **RLS (Row-Level Security):** Política de segurança ao nível da linha que restringe o acesso a registos específicos da base de dados.
- **SMTP (Simple Mail Transfer Protocol):** Protocolo padrão para envio de emails.
- **TOTP (Time-based One Time Password):** Algoritmo que gera passwords descartáveis baseadas no tempo.
- **TypeScript:** Superset do JavaScript com tipagem estática, utilizado no frontend.
- **XSS (Cross-Site Scripting):** Ataque que injeta scripts maliciosos em páginas web.

Obrigado

Ph0enix



Github



LinkedIn



Website